Praktikum Sistem Komunikasi Analog
Pendahuluan Octave/Matlab

Disusun oleh:

Ginanjar S. Adi & Slameta
D-IV Teknik Telekomunikasi
Politeknik Negeri Bandung

Sumber:

Introduction to Octave by Dr. P.J.G. Long
Department of Engineering
University of Cambridge

Tujuan

1. Mampu melakukan dasar operasi kalkulasi, grafik plot, dan menuliskan program sederhana pada software Octave/Matlab
2. Mampu menganalisa parameter dan argument yang ada pada program sederhana

Materi Pendahuluan

Octave dan Matlab digunakan secara luas oleh para ilmuan dan *engineer* baik di dunia industri maupun akademisi untuk melakukan komputasi numerik, serta untuk pengembangan dan menguji algoritma matematis. Software ini dirancang khusus untuk melakukan perhitungan atau memecahkan persamaan matriks. Untuk menyokong pembelajaran yang ideal pada modul ini, jika ada perbedaan fungsi antara Octave dan Matlab akan diberikan keterangan dan dijabarkan secara komprehensif.

1. Kalkulasi sederhana

Penggunaan Octave paling sederhana adalah dengan mengetikkan operasi matematika pada CLI seperti menggunakan kalkulator pada umumnya.

Contoh:

octave:##> 3 + 4

lalu tekan **enter/return** pada prompt, kita akan secara langsung mendapatkan hasil

ans = 7

Operator matematika dasar yang biasa digunakan adalah + - * / dan ^ (digunakan untuk ‘eksponen dari’ e.g. 2^3=8). Urutan utama yang dievaluasi terlebih dahulu adalah **( )**, eksponen^, perkalian* dan pembagian/, penjumlahan+ dan pengurangan-.

2. Fungsi bawaan (*built-in function*)

Octave juga menyediakan semua fungsi matematika standar, seperti ditunjukkan pada Tabel 1. Untuk memanggil fungsi-fungsi (methods) ini dapat dilakukan dengan cara menuliskan:

nama_fungsi(arg)

octave:##> exp(1)

ans = 2.7183

Untuk contoh penulisan ekspresi yang lebih Panjang, seperti $1.2\sin(40° + \ln(2.4^2))$ kita dapat mengetikkan:

octave:##> 1.2 * sin(40*pi/180 + log(2.4^2))

ans = 0.76618

Tabel 1. Fungsi dasar matematika

`cos`	Cosine of an angle (in radians)
`sin`	Sine of an angle (in radians)
`tan`	Tangent of an angle (in radians)
`exp`	Exponential function (e^x)
`log`	Natural logarithm (NB this is $\log_e$, not $\log_{10}$)
`log10`	Logarithm to base 10
`sinh`	Hyperbolic sine
`cosh`	Hyperbolic cosine
`tanh`	Hyperbolic tangent
`acos`	Inverse cosine
`acosh`	Inverse hyperbolic cosine
`asin`	Inverse sine
`asinh`	Inverse hyperbolic sine
`atan`	Inverse tangent
`atan2`	Two-argument form of inverse tangent
`atanh`	Inverse hyperbolic tangent
`abs`	Absolute value
`sign`	Sign of the number (-1 or $+1$)
`round`	Round to the nearest integer
`floor`	Round down (towards minus infinity)
`ceil`	Round up (towards plus infinity)
`fix`	Round towards zero
`rem`	Remainder after integer division

3. Variable

Ketika kita ingin menyimpan atau menggunakan lagi suatu nilai pada kalkulasi tertentu, Octave mengizinkan kita untuk mendefinisikan dan memberikan nama terhadap suatu variable (tipe data variable tidak perlu didefinisikan).

octave:##> A = 180/90

A = 2

4. Memuat dan menyimpan data (*load and save*)

Ketika kita keluar dari Octave, maka kita akan kehilangan semua variable yang telah dibuat. Jika kita ingin menyimpan suatu sesi pekerjaan dan nanti memuatnya kembali, kita dapat mengetikkan pada prompt:

octave:##> save nama_workspace

perintah ini akan menyimpan workspace secara keseluruhan pada direktori kita dengan file berekstensi *.mat (**nama_workspace.mat**). Untuk memuat kembali workspace yang telah kita simpan, ketikan perintah:

octave:##> load nama_workspace

5. Vektor dan matriks

Banyak cara untuk mendefinisikan vector dan matriks, diantaranya adalah:

octave:##> a=[1 4 5]

a = 1 4 5

octave:##> b=[2,1,0]

b = 2 1 0

octave:##> c=[4;7;10]

c =

4
7
10

Suatu daftar angka yang dipisahkan dengan spasi atau koma, didalam kurung kotak (*square brakets* []), menunjukkan suatu vektor baris. Sedangkan angka-angka yang

dipisahkan oleh titik koma (*semicolons* atau *carriage returns*) merepresentasikan suatu vektor kolom.

Shortcut yang berguna untuk membangun vektor dengan inkremen tertentu adalah dengan menggunakan simbol titik dua ':', Seperti dalam contoh berikut ini:

octave:##> e=2:6

e = 2 3 4 5 6

Simbol titik dua memerintahkan Octave untuk membuat suatu vector mulai dari angka pertama dan menghitung hingga (termasuk) angka kedua.
Angka ketiga dapat juga dicantumkan diantara angka pertama dan ke dua, a : b : c. Angka yang berada ditengah menentukan kenaikan antara tiap elemen pada vector.

octave:##> e=2:0.3:4

e = 2.0000 2.3000 2.6000 2.9000 3.2000 3.5000 3.8000

Perlu diperhatikan bahwa hal yang terjadi pada contoh diatas tidak dapat mencapai persis angka terakhir, akan tetapi ia akan menghasilkan semua angka yang tidak melebihi angka tersebut. Tabel 2 menunjukkan beberapa fungsi untuk membentuk suatu vektor dan matriks.

Tabel 2. Fungsi vektor dan matriks

`zeros(`M, N`)`	Create a matrix where every element is zero. For a row vector of size n, set $M = 1, N = n$
`ones(`M, N`)`	Create a matrix where every element is one. For a row vector of size n, set $M = 1, N = n$
`linspace(`$x1, x2, N$`)`	Create a vector of N elements, evenly spaced between $x1$ and $x2$
`logspace(`$x1, x2, N$`)`	Create a vector of N elements, logarithmically spaced between 10^{x1} and 10^{x2}

Individual elemen dapat dirujuk dengan menggunakan tanda kurung (),

octave:##> a=[1:2:6 -1 0]

a = 1 3 5 -1 0

kita bisa mendapatkan elemen ke-3 dari vector a dengan mengetikan:

octave:##> a(3)

ans = 5

simbol titik dua ‘:’ bisa juga digunakan untuk menentukan rentang angka untuk mendapatkan beberapa elemen pada satu waktu.

octave:##> a(3:5)

ans = 5 -1 0

octave:##> a(1:2:5)

ans = 1 5 0

Menyimpan daftar angka ke dalam suatu vektor memungkinkan Octave untuk menggunakan fiturnya dalam melakukan perhitungan. Octave dapat melakukan perkalian semua angka dalam suatu vektor dengan sebuah konstanta (*broadcasting*). Dengan contoh vektor a dikalikan dengan 2, kita dapat menuliskan perintah:

octave:##> a * 2

ans = 2 6 10 -2 0

Mengalikan dua vektor Pada Octave mengikuti aturan perkalian matriks, tidak melakukan perkalian elemen demi elemen. Jika kita ingin melakukan perkalian per elemen, Octave mendefinisikan operator . * dan ./, misalnya

$$\begin{pmatrix} a_1 \\ a_2 \\ a_3 \end{pmatrix} .* \begin{pmatrix} b_1 \\ b_2 \\ b_3 \end{pmatrix} = \begin{pmatrix} a_1 b_1 \\ a_2 b_2 \\ a_3 b_3 \end{pmatrix}$$

Perlu dicatat bahwa penggunaan ‘.’ didepan setiap simbol aritmatika menandakan operasi elemen demi elemen.

octave:##> b=[1 2 3 4 5];

octave:##> a.*b

ans = 1 6 15 -4 0

octave:##> b .^ 2

ans = 1 4 9 16 25

octave:##> 2 .^ b

ans = 2 4 8 16 32

6. Grafik plot

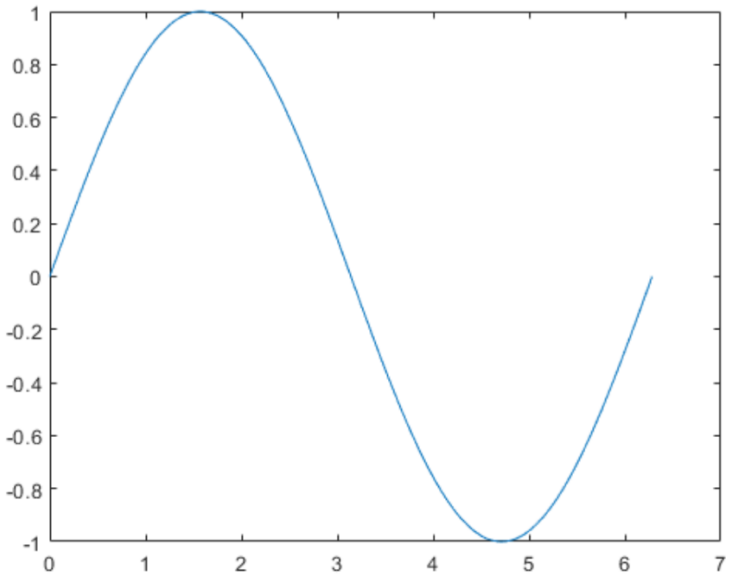
Octave memiliki fasilitas handal untuk memplot grafik melalui *package* GNUPLOT, namun beberapa fitur plot dibatasi jika dibandingkan dengan MATLAB. Perintah dasar untuk melakukan plot adalah

plot (x, y)

dimana x dan y adalah koordinatnya. Jika x dan y masing-masing adalah suatu konstanta angka, maka Octave akan melakukan plot sebuah titik. Biasanya yang sering kita gunakan adalah vektor, hal ini akan menyebabkan semua poin yang diberikan oleh kedua vektor akan diplot pada suatu bidang. Contoh:

octave:##> x = 0:pi/100:2*pi;
octave:##> y = sin(x);
octave:##> plot(x,y)

syntax diatas menunjukkan x sebagai vektor nilai spasi linear antara 0 dan 2π dengan inkremen π/100 di antara nilai tersebut. Variable y sebagai nilai sinus x. Plot garis data.



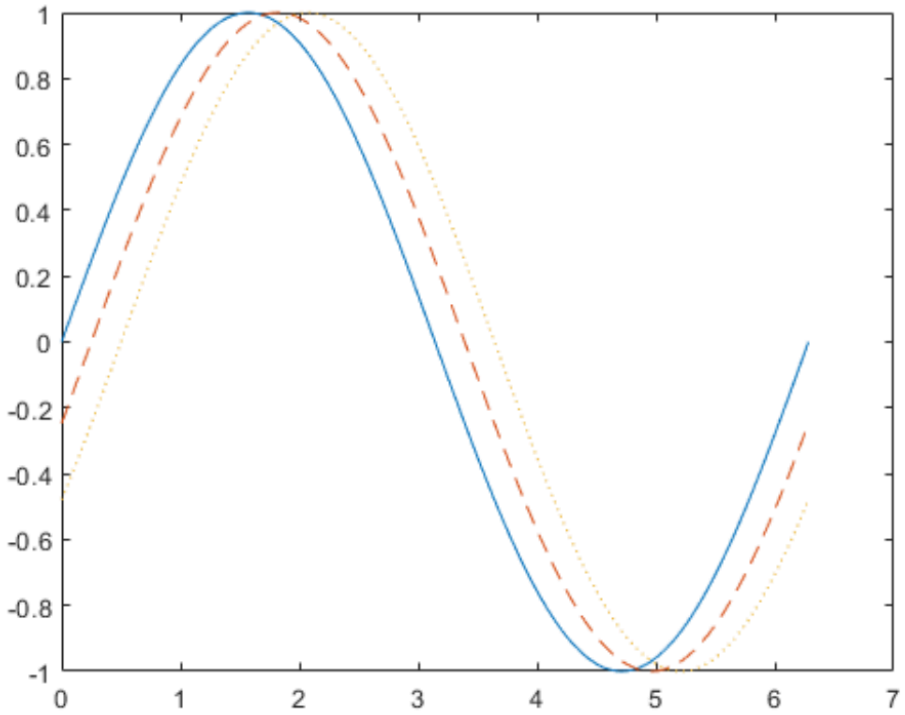
Perlu diperhatikan bahwa, jika x dan y merupakan vektor maka keduanya harus memiliki dimensi yang sama.

Contoh berikutnya, kita akan melakukan plot 3 gelombang sinusoidal dengan sedikit pergeseran fasa diantara setiap gelombangnya. Kita gunakan *dashed line* '--' untuk sinyal

kedua dan *dotted line* ‘:’ untuk sinyal ketiga (‘:’ *dotted line* hanya nisa digunakan pada Matlab).

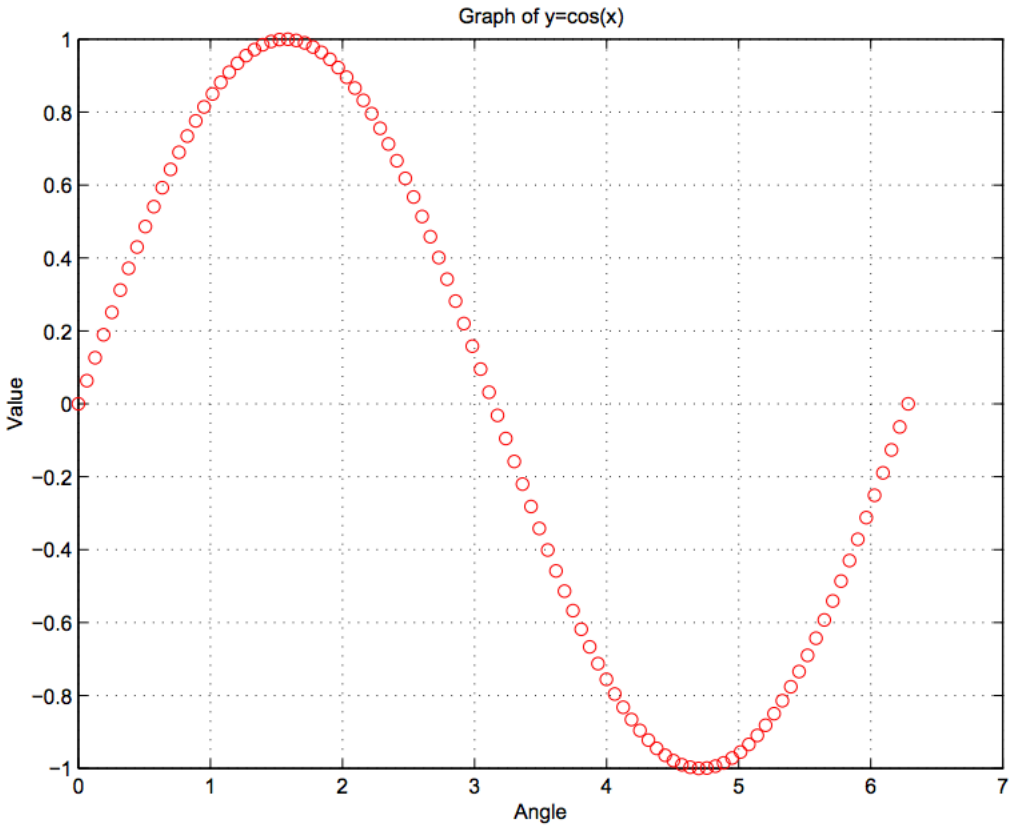
```
octave:##> x = 0:pi/100:2*pi;
octave:##> y1 = sin(x);
octave:##> y2 = sin(x-0.25);
octave:##> y3 = sin(x-0.5);

octave:##> figure
octave:##> plot(x,y1,x,y2,'--',x,y3,':')
```



Kita dapat memilih warna dan ragam garis pada perintah plot untuk memodifikasi tampilan. Tabel 3 menunjukkan parameter yang mungkin digunakan untuk Octave dan Matlab. Untuk mencantumkan judul, label axis, dan grid pada grafik kita bisa menggunakan perintah sbb:

```
octave:##> angles=linspace(0,2*pi,100);
octave:##> y=sin(angles);
octave:##> plot(angles, y);
octave:##> plot(angles, y, ’ro’)
octave:##> title(’Graph of y=sin(x)’)
octave:##> xlabel(’Angle’)
octave:##> ylabel(’Value’)
octave:##> grid on
```



Tabel 3. Warna dan macam-macam simbol pada perintah plot (gunakan help plot untuk melihat argument dan parameter fungsi plot); N.B. † hanya terdapat pada Matlab

`w`	whitew	`.`	point	`-`	solid
`m`	magenta	`o`	circle	`:`	dotted†
`c`	cyan	`x`	x-mark	`-.`	dashdot†
`r`	red	`+`	plus	`--`	dashed†
`g`	green	`*`	star		
`b`	blue	`s`	square†		
`y`	yellow†	`d`	diamond†		
`k`	black†	`v`	triangle (down)†		
		`^`	triangle (up)†		
		`<`	triangle (left)†		
		`>`	triangle (right)†		
		`p`	pentagram†		
		`h`	hexagram†		

Percobaan

Bangkitkan sinyal sine dan cosine masing-masing dengan atribut:

- V = 1 volt, f = 1000 Hz, t = 0 sampai 1 second, fs = 10000, warna garis magenta untuk gelombang sine
- V = 2 volt, f = 500 Hz, t = 0 sampai 1 second, fs = 10000, warna garis green untuk gelombang cosine

1. Plot kedua gelombang tersebut pada satu figure, modifikasi label sumbu y dan x masing-masing menjadi Voltage (Volt) dan Time (Second).
2. Plot untuk t mulai dari 0 sampai 1 second bandingkan dengan ketika t mulai dari 0.01 sampai 0.02 second.
3. Ubahlah fs = 1000 untuk masing-masing sinyal; Plot seperti langkah 2 dan analisa perbedaan antara fs = 10000 dan fs = 1000.

Hint: manfaatkan fungsi “help” pada command window. Mungkin fungsi “linspace” dan “xlim” akan berguna.